

HLS Project: Image Texture Classification

Rodrigo Olmos, Hugo Bueno and Emilio Pérez

High-Level Description of Systems

June 2024

High-Level Synthesis: Optimization in Vitis HLS

Introduction

The aim of this project is to optimize an image classification program for use in an FPGA. To achieve this, the first step was to port the code from Matlab to C so Vitis HLS could be used to optimize the system at a high level of abstraction. This optimization is achieved both by modifying the C code directly as well as by applying certain directives that are considered when the C code is synthesized into a hardware description. Of these directives, the most used ones are array partitioning, loop unrolling and loop flattening, described in the following paragraphs.

Directives in Vitis HLS

Array partitioning is a technique in hardware design where an array is divided into smaller sub-arrays that can be processed in parallel. This can be done in different ways, such as block partitioning, cyclic partitioning, or complete partitioning. The main advantage of array partitioning is that it allows multiple sub-arrays to be processed at the same time, which increases overall speed. Also, smaller sub-arrays mean faster data access, improving performance, and making better use of hardware resources by spreading the workload. However, this technique also makes the design more complex because it needs extra control logic to manage the sub-arrays. It can also use more memory resources, which can be a problem if resources are limited. If the partitioning is not done well, some sub-arrays might not be used efficiently, leading to wasted resources.

Loop unrolling is a technique where the iterations of a loop are expanded into a series of repeated instructions, reducing the overhead of loop control and increasing parallelism in the loop body. This reduces the overhead related to loop control, such as incrementing counters and checking conditions, which speeds up execution. It also allows more instructions to run in parallel because there are fewer dependencies within the loop. This can improve performance by making better use of the hardware pipeline and reducing the number of branch instructions. Not all loops can be unrolled effectively, especially if the number of iterations is unknown at compile time or if the loop has complex control flow.

Loop flattening is the process of turning nested loops into a single loop, simplifying the control flow and potentially exposing more parallelism. This can make the control logic simpler and the hardware design easier to implement and debug. It also creates more opportunities for parallel execution by reducing dependencies between loops, which can improve performance. However, flattening loops requires complex index calculations to map the original nested structure to a single loop, adding overhead. Not all nested loops can be flattened, especially if they have interdependencies or different iteration counts.

Local Binary Pattern Features block

Local Binary Patterns (LBP) is a method used in image classification to analyze texture by examining each pixel and its surrounding 8-pixel neighborhood. Each neighboring pixel is compared to the central pixel, generating an 8-bit binary value that is then converted to a decimal value. The output of the LBP process is a histogram with 256 bins, representing the frequency of each LBP code in the image, which serves as a characteristic feature for image classification and is used by the following block, the system vectoring machine block (SVM).

For this block, three different designs were written down in code, and for each of them, several values for the directives were tested.

Code optimization

1. Standard

The standard version of the code is used to perform Local Binary Pattern (LBP) feature extraction on an image. This method helps to capture the texture of the image. The code goes through each pixel in the image, except for the border pixels, to avoid errors. For each pixel, it compares the pixel's intensity to its eight neighbors. These comparisons create an 8-bit binary number, the LBP code, where each bit shows if the center pixel's intensity is greater than or less than its neighbor's intensity. This binary number is then turned into a decimal value and used to update a histogram that counts how often each LBP code appears in the image.

After the histogram is filled, the code calculates the sum of the squares of the histogram values to find the L2 norm, which measures the histogram's magnitude. The final step is to normalize the histogram by dividing each value by the L2 norm, resulting in a feature vector. This vector can be used for image analysis or pattern recognition. However, this standard approach involves many nested loops and memory accesses, which can be slow, especially for large images.

The two other versions of the code, called *pixels_splitedx2* and *pixels_splitedx4* use two and three times the value of the SPLIT macro of the original, which was 32. This implies further parallelization of the operations performed on the array, resulting in a significant reduction in latency, at the cost of more LUTs necessary for the additional parallel operations. These effects are quantified and collected in Figure 1 and graphically represented in Figure 2.

Latency				
		solution1	pixels_splitedx2	pixels_splitedx4
Latency (cycles)	min	80299	49548	33672
	max	80299	49548	33672
Latency (absolute)	min	0.803 ms	0.495 ms	0.337 ms
	max	0.803 ms	0.495 ms	0.337 ms
Interval (cycles)	min	80300	49549	33673
	max	80300	49549	33673

Utilization Estimates				
		solution1	pixels_splitedx2	pixels_splitedx4
BRAM_18K	1	1	1	
DSP	1	1	1	
FF	944	952	907	
LUT	1288	1392	1656	
URAM	0	0	0	

Figure 1. Screenshot of a table from Vitis HLS showing the latency and number of components used by different solutions based on the standard code of the LBP module.

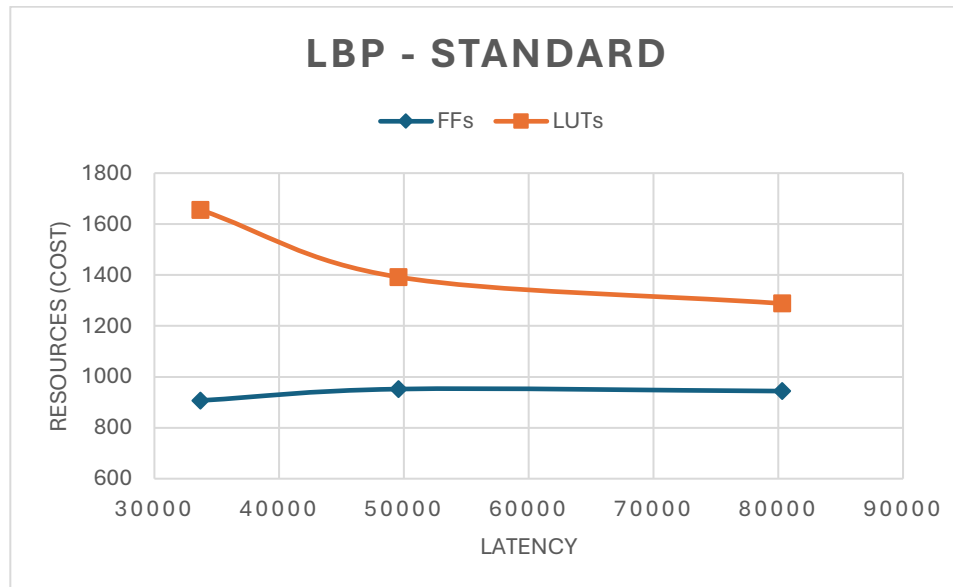


Figure 2. Graph relating the latency and number of components used by the various solutions based on the standard code of the LBP module.

2. Improvement #1

The first improvement makes the standard version faster and more efficient. One big change is splitting the image into smaller chunks, the number of which is determined by the SPLIT macro, which was set to 32. This way, the image can be processed in parts, making it easier to handle large images and improving memory use. The code now uses a function, `extract_histogram()`, to process each chunk separately, updating a local histogram for each split. This function handles a smaller part of the image and

computes the LBP codes and histogram updates locally, and makes the possibility of parallelization of the feature calculation process more explicit

Another key improvement is how the histogram values are summed. Instead of calculating the sum of squares in one go, the code uses a reduction technique where histogram products are computed and summed in pairs. This means adding the histogram products two at a time, reducing the number of sums needed in each step until only one sum is left. This makes the computation simpler and allows for more parallel processing, which is faster. These changes reduce memory use, improve data handling, and make the code run faster, making it better for high-level synthesis tools that can use parallel processing.

The two other versions of the code, called `pixels_splitedx2` and `pixels_splitedx4` use 2 and three times the value of the `SPLIT` macro of the original, which was 32. This implies further parallelization of the operations performed on the array, resulting in a significant reduction in latency, at the cost of more LUTs necessary for the additional parallel operations. These effects are quantified and collected in Figure 3 and graphically represented in Figure 4.

Timing				
Clock		solution1	pixels_splitedx2	pixels_splitedx4
ap_clk	Target	10.00 ns	10.00 ns	10.00 ns
	Estimated	6.949 ns	6.949 ns	6.996 ns

Latency				
		solution1	pixels_splitedx2	pixels_splitedx4
Latency (cycles)	min	71609	43385	29273
	max	605945	573689	557561
Latency (absolute)	min	0.716 ms	0.434 ms	0.293 ms
	max	6.059 ms	5.737 ms	5.576 ms
Interval (cycles)	min	71610	43386	29274
	max	605946	573690	557562

Utilization Estimates			
	solution1	pixels_splitedx2	pixels_splitedx4
BRAM_18K	3	3	3
DSP	1	1	1
FF	1620	1542	1456
LUT	2853	3135	3801
URAM	0	0	0

Figure 3. Screenshot of a table from Vitis HLS showing the latency and number of components used by different solutions based on the code of the first improvement to the LBP module.

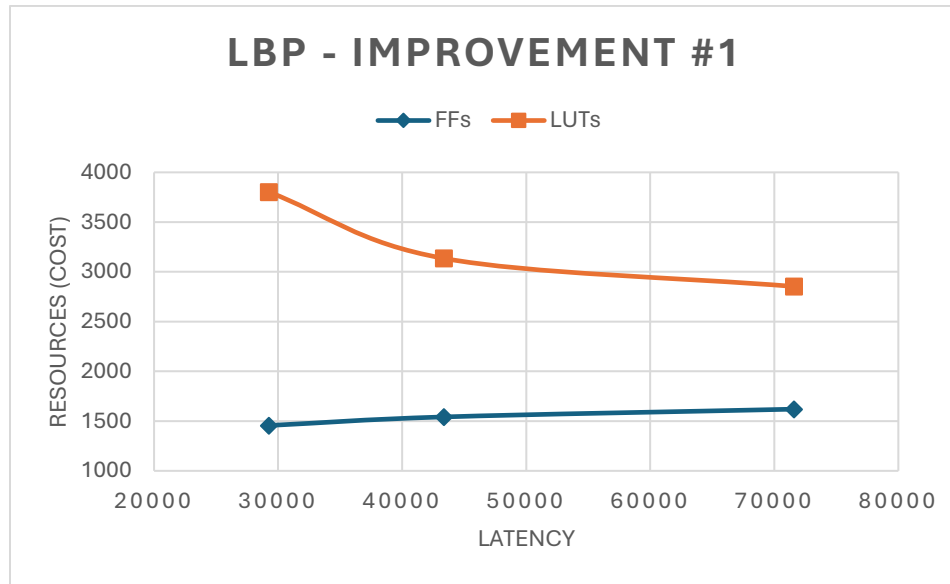


Figure 4. Graph relating the latency and number of components used by the various solutions on the code of the first improvement to the LBP module.

3. Improvement #2

The second improvement builds on the first improvement and makes the code even more efficient for hardware use. One big addition is a fast inverse square root function, `sqrt_hls()`, which quickly approximates the square root. This makes the normalization step much faster.

Improvement 2 also changes how the LBP codes are extracted by removing the need for a separate neighborhood array. Instead, the code calculates the neighbor indices as needed within the `extract_histogram()` function, reducing memory use and improving speed. The image is still processed in slices, but the method of storing and accessing these slices is optimized. Local arrays are used to store parts of the image, ensuring efficient memory access and allowing for parallel processing.

Additionally, the process of combining and accumulating the histogram values is streamlined. Histograms from all chunks are combined, and their squared values are summed more efficiently. This reduces the number of passes over the data, making the computation faster. By focusing on efficient memory use, fast critical operations, and enhanced parallel processing, Improvement 2 makes the code highly efficient and suitable for high-level synthesis tools that can use parallelism and pipelining.

These changes in the code by themselves almost halve the time it takes to carry out the computation. However, when the directives, shown in Figure 5, which partition all the arrays and unroll all the loops are added, the time taken to compute becomes 21 times smaller, with the amount of look-up tables used being multiplied by 14 and the

number of flip flops used only increasing by a factor of 4. These effects are quantified and collected in Figure 6 and graphically represented in Figure 7. This last version represents a much larger jump in performance gains and resources used with respect to the previous improvements, but still the number of LUTs used is only half of those available in the FPGA used for design, making it still technically viable, with the only possible concerns being the reduced space left to other potential functionalities that may be needed or an increased power consumption.

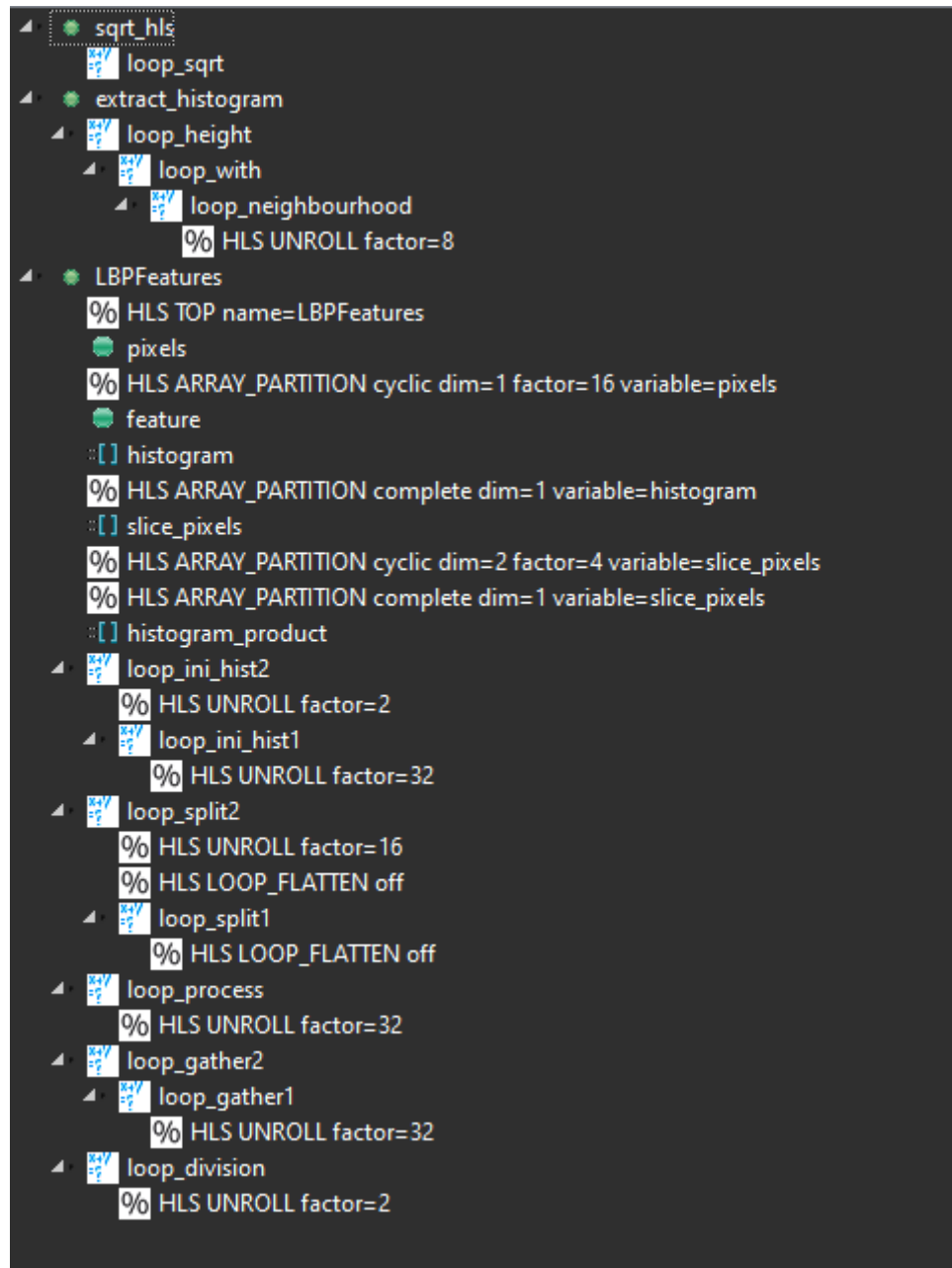


Figure 5. Screenshot from Vitis HLS showing the directives applied to the code of the second improvement to the LBS module, which is also the definitive version.

Timing					
Clock		solution1	more_and_more	pixels2	pixels4
ap_clk	Target	10.00 ns	10.00 ns	10.00 ns	10.00 ns
	Estimated	7.256 ns	7.256 ns	7.256 ns	7.256 ns

Latency					
		solution1	more_and_more	pixels2	pixels4
Latency (cycles)	min	42072	2010	42073	42073
	max	58200	2514	58201	58201
Latency (absolute)	min	0.421 ms	20.100 us	0.421 ms	0.421 ms
	max	0.582 ms	25.140 us	0.582 ms	0.582 ms
Interval (cycles)	min	42073	2011	42074	42074
	max	58201	2515	58202	58202

Utilization Estimates					
		solution1	more_and_more	pixels2	pixels4
BRAM_18K		192	160	192	192
DSP		6	6	6	6
FF		1992	8459	2028	2010
LUT		4021	55060	4027	4027
URAM		0	0	0	0

Figure 6. Screenshot of a table from Vitis HLS showing the latency and number of components used by different solutions based on the code of the second improvement to the LBP module.

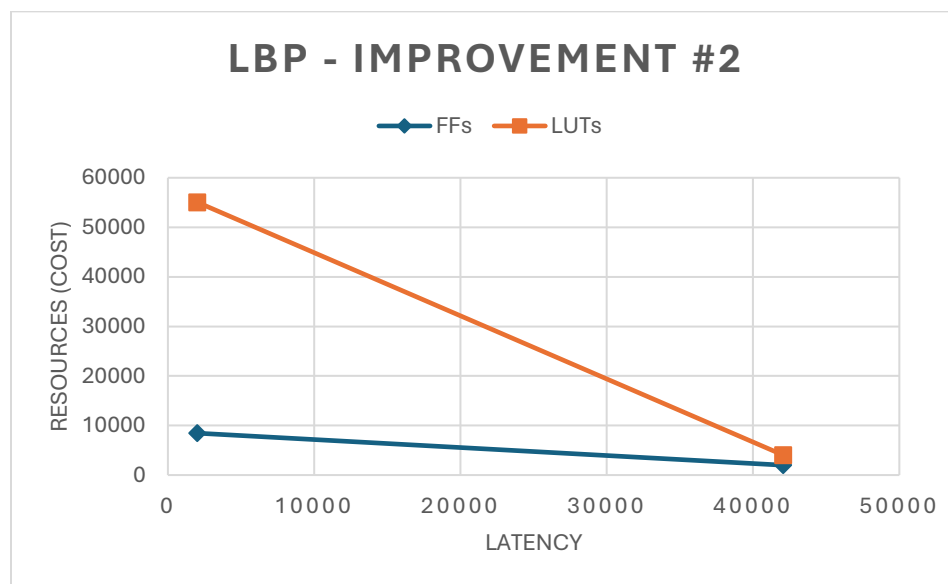


Figure 7. Graph relating the latency and number of components used by the various solutions on the code of the second improvement to the LBP module.

Support Vector Machine block

The SVM function takes an array of input features and several model parameters to predict a class. It normalizes the features using given means and standard deviations, then scales them and accumulates the results with the model's coefficients. For each type, it adds a bias term and checks which type has the highest value. The function then sets the type with the highest value as the prediction. This method processes the input features and uses the SVM model to classify them based on the highest score.

There is only one version of the code for this part, with the improvements quantified and collected in Figure 9 and graphically represented in Figure 10 corresponding to the directives shown in Figure 8. The basic solution includes no directives.

Loop_unroll only unrolls the for loop in the code and is actually outside the pareto front. The other two include more and more array partitions on the data.

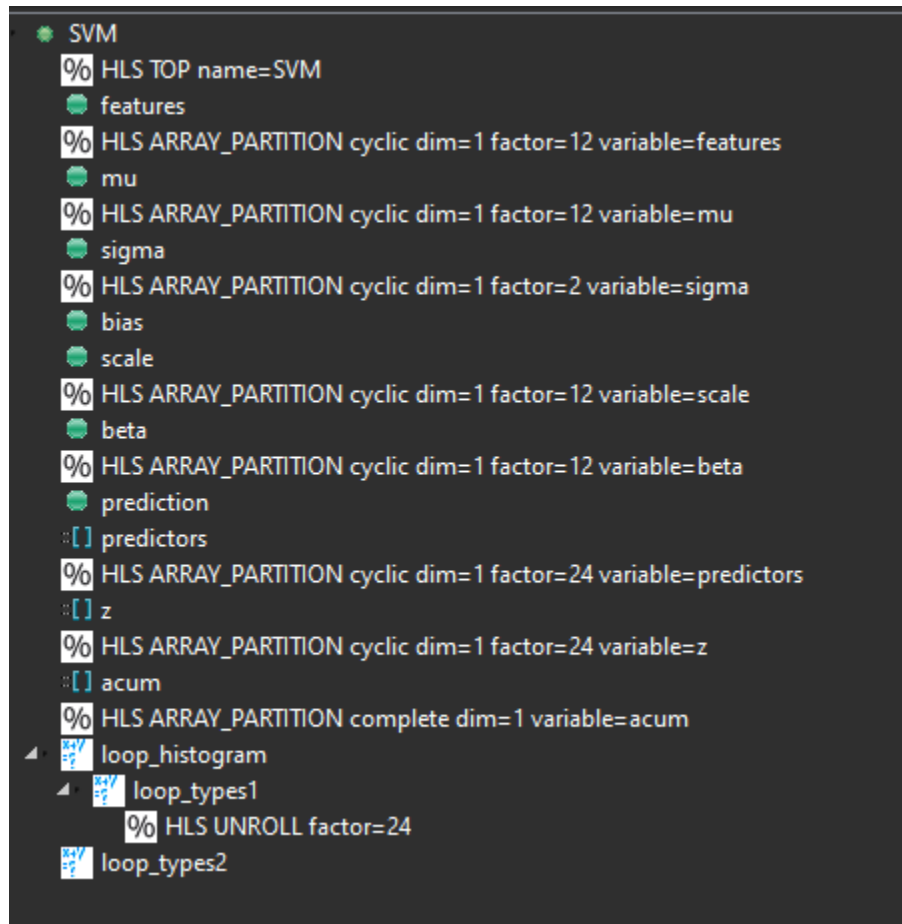


Figure 8. Screenshot from Vitis HLS showing the directives applied to the code of the SVM module.

Latency					
		basic	loop_unroll	unroll_internal_partitions	unroll_all_partitions
Latency (cycles)	min	6312	3233	3193	1653
	max	6312	3233	3193	1653
Latency (absolute)	min	63.120 us	32.330 us	31.930 us	16.530 us
	max	63.120 us	32.330 us	31.930 us	16.530 us
Interval (cycles)	min	6313	3234	3194	1654
	max	6313	3234	3194	1654

Utilization Estimates				
	basic	loop_unroll	unroll_internal_partitions	unroll_all_partitions
BRAM_18K	0	0	0	0
DSP	9	14	14	28
FF	7300	9356	8502	9987
LUT	4163	5943	5536	7196
URAM	0	0	0	0

Figure 9 . Screenshot of a table from Vitis HLS showing the latency and number of components used by the two solutions for the SVM module.

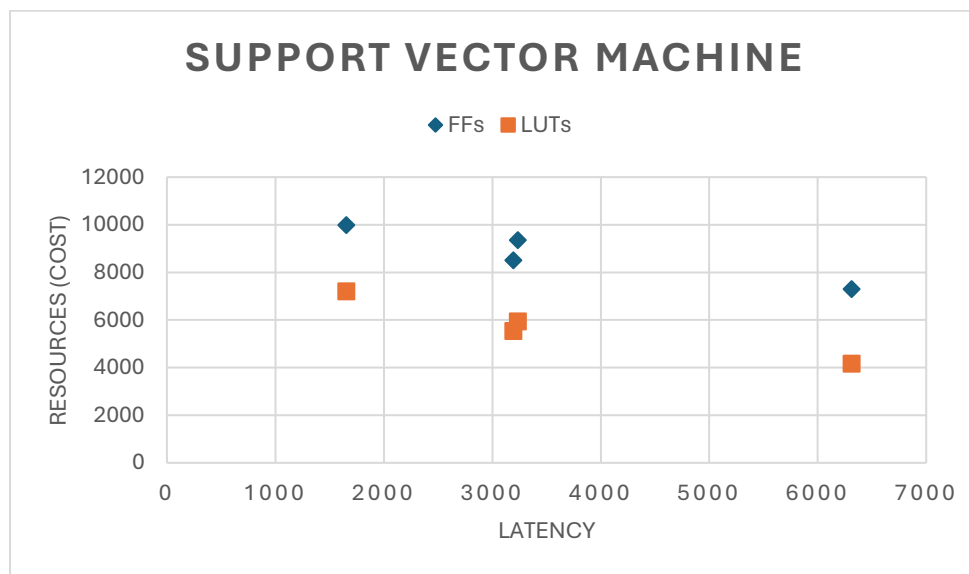


Figure 10. Graph relating the latency and number of components used by the two solutions for the SVM module.

Top – Testbench

The top project combines both previously optimized blocks in order to perform the task required from the system, which in this case will be to classify a given set of images. Since most of the operations are performed by the two previously discussed main blocks, there is no need to optimize this project particularly. Instead, the fastest design points obtained for the previous blocks were combined to form it. The total time it would take to complete the test is estimated to be 39 ms, as can be seen in Figure 11. The accuracy achieved by our version was the same as was achieved with the Matlab one. The combined functions and their applied directives can be seen in Figures Figure 12 and Figure 13.

Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSR	FF	LU*	URAM
▲ classifier				-1.19	3896	3.896E4	-	3897	-	no	160	30	18455	62324	0
▶ LBPFeatures_27_28	II Violation		-	-	2242	2.242E4	-	2242	-	no	160	2	8082	54396	0
▶ classifier_Pipeline_loop_histogram	II Violation		-1.19	-	1592	1.592E4	-	1592	-	no	0	23	8615	6050	0
▶ classifier_Pipeline_loop_types2	II Violation		-	-	57	570.000	-	57	-	no	0	0	249	348	0

Figure 11. Latency achieved and number of components used in the project testbench.

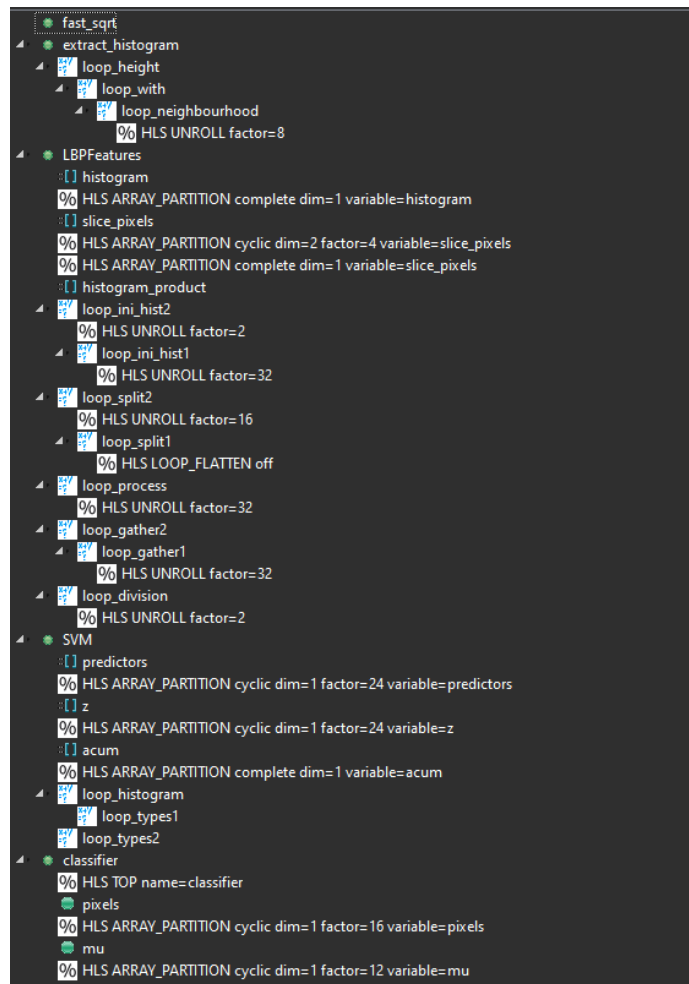


Figure 12. Directives used for the code of the project testbench. Since the original image is too long, it has been split in two, and this is the first half.

```

% HLS ARRAY_PARTITION cyclic dim=2 factor=4 variable=slice_pixels
% HLS ARRAY_PARTITION complete dim=1 variable=slice_pixels
::[] histogram_product
└─ loop_ini_hist2
    % HLS UNROLL factor=2
    └─ loop_ini_hist1
        % HLS UNROLL factor=32
    └─ loop_split2
        % HLS UNROLL factor=16
        └─ loop_split1
            % HLS LOOP_FLATTEN off
    └─ loop_process
        % HLS UNROLL factor=32
    └─ loop_gather2
        └─ loop_gather1
            % HLS UNROLL factor=32
    └─ loop_division
        % HLS UNROLL factor=2
└─ SVM
    ::[] predictors
    % HLS ARRAY_PARTITION cyclic dim=1 factor=24 variable=predictors
    ::[] z
    % HLS ARRAY_PARTITION cyclic dim=1 factor=24 variable=z
    ::[] acum
    % HLS ARRAY_PARTITION complete dim=1 variable=acum
    └─ loop_histogram
        loop_types1
        loop_types2
└─ classifier
    % HLS TOP name=classifier
    pixels
    % HLS ARRAY_PARTITION cyclic dim=1 factor=16 variable=pixels
    mu
    % HLS ARRAY_PARTITION cyclic dim=1 factor=12 variable=mu
    sigma
    % HLS ARRAY_PARTITION cyclic dim=1 factor=2 variable=sigma
    bias
    scale
    % HLS ARRAY_PARTITION cyclic dim=1 factor=12 variable=scale
    beta
    % HLS ARRAY_PARTITION cyclic dim=1 factor=12 variable=beta
    prediction
    ::[] features
    % HLS ARRAY_PARTITION cyclic dim=1 factor=12 variable=features

```

Figure 13. Directives used for the code of the project testbench. Since the original image is too long, it has been split in two, and this is the second half.

Pareto fronts

The variation in BRAMs and DSPs is none or negligible, so the Pareto fronts are done using only LUTs and FFs as resources. The Pareto front formed by the combination of the three versions of the LBP code and their various solutions is summarized in Figure 14. The fourth, fifth and seventh solutions counting from the left, which correspond to solution1 from the second code improvement, pixel_split1 from the first code improvement, and solution 1 from the first code improvement, respectively, lie outside the Pareto front, since they use more components than other solutions which also have lower latency.

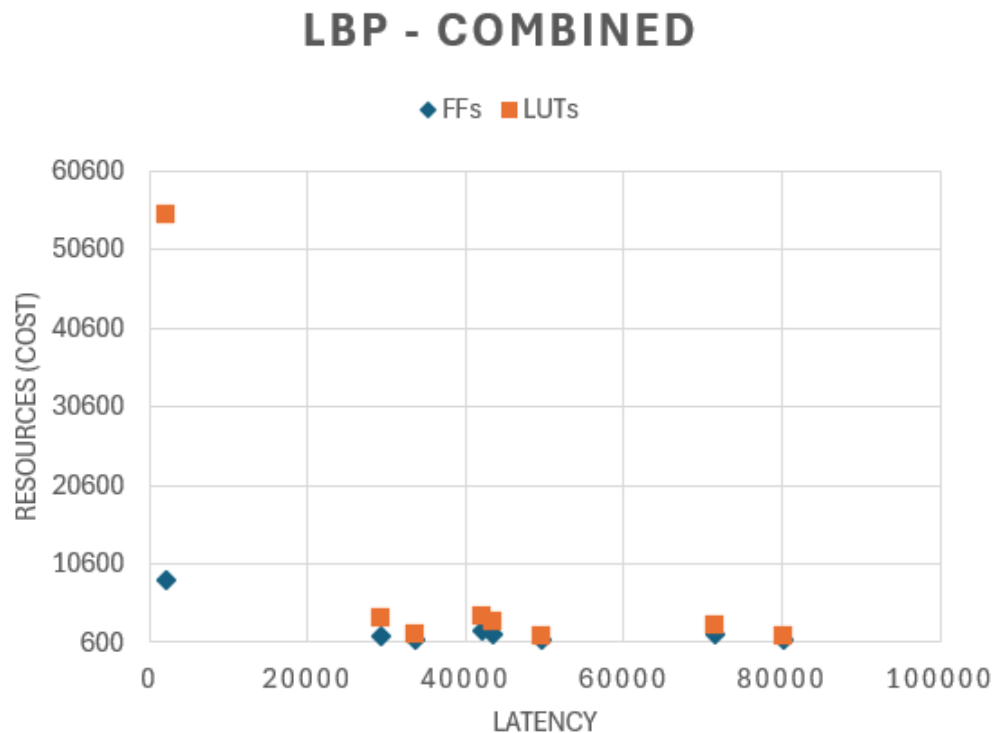


Figure 14. Graph relating the latency and number of components used by all the studied solutions of the LBP module.

The Pareto front corresponding to the SVM module is represented in Figure 10.

System C

The construction of the algorithm designed for the previous part can be seen from another approach. When creating it for SystemC, the user can better perceive the partition into its different modules and communications. All the files for this part can be found attached at the end of the document. A diagram of the whole system can be found in Figure 15.

The **system's heart resides in the Classifier main module**, that combines both the Local Binary Pattern and System Vectoring Machine algorithms. A Testbench (together with its main file) connects it to the rest of the system.

1. **The channel between the images and the Classifier module has an intermediate memory** in order to transform the images into the buffer array. Later on, this array is transmitted to the Classifier so as to process it.
2. The model parameters are given to the Classifier through the inclusion of "model.h" file, which has its values already written from a Matlab modification we made on our own.
3. Finally, the Classifier sends the final prediction with a simple signal to the output port.

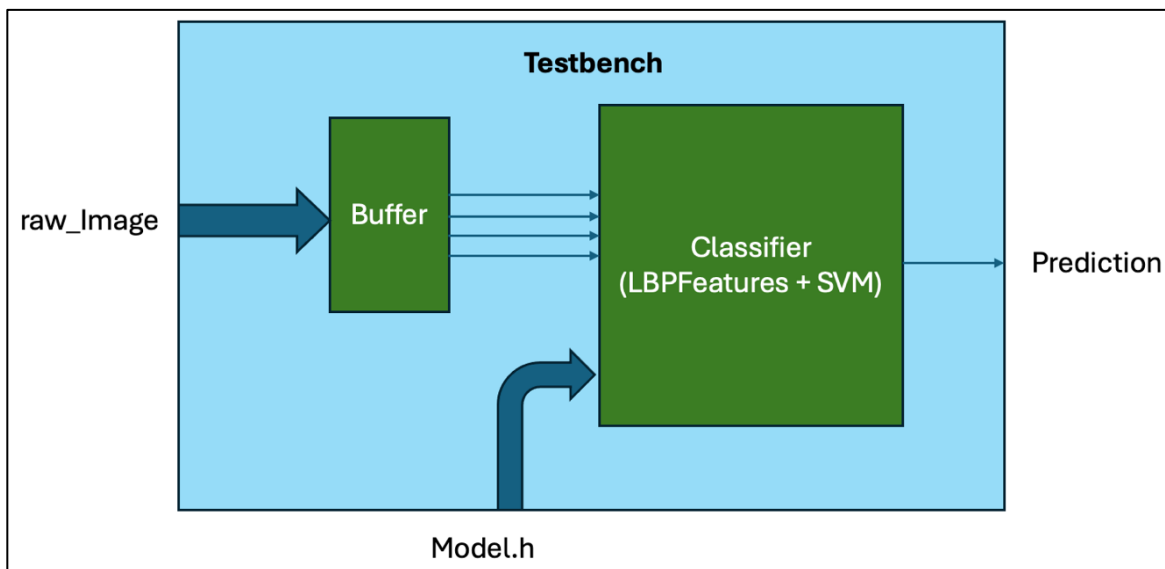


Figure 15. Diagram showing the SystemC components and their interconnections.

Classifier

The classifier module combines, as it was pointed out before, both the LBPFeatures and SVM algorithms (based on the most optimized versions of the HLS part) into a single one. However, this very module has several processes attached:

- **Classify():** thread that waits for a start in order to carry out the rest of the processes (i.e. behaving synchronously).
- **Extract_histogram():** main part of LBPFeatures algorithm, which delivers the core texture of the image through an histogram.
- **LBPFeatures():** besides from calling the previous process, this method splits the pixels for a higher parallelization and normalizes the histogram.
- **SVM():** method that makes the proper classification of the image according to its histogram and the parametres of the trained model.
- **Fast_sqrt():** own-made square root algorithm, based on Newton-Raphson's.

Testbench

The system behaves in a synchronous manner: a start, a reset and a clock signal impose this behaviour. There are two main processes described:

- **Load_test_image():** this is the reading procedure for images, as well as its transformation into an array and deployment towards the classifier module.
- **Run():** thread that executes the whole algorithm, from the load of the image to its classification.

Code Files

Below, the .h and .cpp files used for this part are attached.

```

1  //////////////////////////////////
2  // CLASSIFIER.H //
3  //////////////////////////////////
4
5  /* UNIFICATION OF LBPFeatures + SVM Blocks */
6
7  #ifndef CLASSIFIER_H
8  #define CLASSIFIER_H
9
10 #include <systemC.h>
11 #include <cstdint>
12
13 #define IMG_WIDTH 28
14 #define IMG_HEIGHT 28
15 #define HISTOGRAM_WIDTH 256
16 #define SPLIT 2
17 #define TYPES 10
18
19
20 SC_MODULE(Classifier) {
21     sc_in<bool> clk;
22     sc_in<bool> reset;
23     sc_in<bool> start;
24     sc_in<uint8_t> pixels[IMG_WIDTH * IMG_HEIGHT];
25     sc_out<uint8_t> prediction;
26
27
28     SC_CTOR(Classifier) {
29         SC_THREAD(classify);
30         sensitive << clk.pos();
31         async_reset_signal_is(reset, true); //
32     }
33
34     void classify();
35
36     void LBPFeatures(const sc_in<uint8_t> pixels[IMG_HEIGHT * IMG_WIDTH], float feature[HISTOGRAM_WIDTH]);
37
38     void SVM(float features[HISTOGRAM_WIDTH], uint8_t *prediction);
39
40     float fast_sqrt(float number);
41 };
42
43 #endif // CLASSIFIER_H

```



```

1  //////////////////////////////////
2  // CLASSIFIER.CPP //
3  //////////////////////////////////
4
5  #include "classifier.h"
6  #include "model.h" //Introduce SVM_Model parameters
7  #include <cmath>
8
9  void Classifier::classify() {
10     uint8_t temp;
11     while (true) {
12         wait();
13
14         //Synchronous behavior
15         if (start.read()) {
16             float features[HISTOGRAM_WIDTH];
17
18             LBPFeatures(pixels, features);
19
20             SVM(features, &temp);
21
22             prediction.write(temp);
23         }
24     }
25 }
26
27 void extract_histogram(uint8_t pixels[IMG_HEIGHT * IMG_WIDTH / SPLIT + 2 * IMG_WIDTH], uint16_t histogram[HISTOGRAM_WIDTH], uint8_t length){
28
29     uint8_t transform;
30     uint8_t i, j, kk;
31
32     loop_height: for (i = 1; i < length; i++){
33         loop_width: for (j = 1; j < IMG_WIDTH - 1; j++){
34
35             transform = 0;
36             loop_neighbourhood: for(kk = 0; kk < 8; kk++){
37
38                 // Associate offset according to neighbourhood position
39                 int8_t row_offset = (kk == 1 || kk == 2 || kk == 3) ? -1 : (kk == 5 || kk == 6 || kk == 7) ? 1 : 0;
40                 int8_t col_offset = (kk == 3 || kk == 4 || kk == 5) ? -1 : (kk == 7 || kk == 0 || kk == 1) ? 1 : 0;
41
42                 // Add the correspondant binary weight if the neighbor is higher
43                 transform += (pixels[i * IMG_WIDTH + j] < pixels[(i + row_offset) * IMG_WIDTH + (j + col_offset)]) ? (1 << kk) : 0;
44
45             }
46
47             histogram[transform]++;
48         }
49     }
50 }
51
52 void Classifier::LBPFeatures(const sc_in<uint8_t> pixels[IMG_HEIGHT * IMG_WIDTH], float feature[HISTOGRAM_WIDTH]) {
53     uint16_t i;
54     uint8_t j;
55
56     uint16_t histogram[SPLIT][HISTOGRAM_WIDTH];
57     uint8_t slice_pixels[SPLIT][IMG_HEIGHT*IMG_WIDTH/SPLIT+2*IMG_WIDTH];
58     uint32_t histogram_product[HISTOGRAM_WIDTH];
59     uint32_t acum = 0;
60
61     // Initialise histogram to 0
62     loop_ini_hist2: for (i = 0; i < HISTOGRAM_WIDTH; i++) {
63         loop_ini_hist1: for (j = 0; j < SPLIT; j++) {
64             histogram[j][i] = 0;
65         }
66     }
67
68     uint16_t slice_size = IMG_HEIGHT * IMG_WIDTH / SPLIT;
69     uint16_t extended_slice_size = slice_size + 2 * IMG_WIDTH;
70
71     // Split pixels
72     loop_split2: for (i = 0; i < extended_slice_size; i++) {
73         loop_split1: for (j = 0; j < SPLIT; j++) {
74             if (i < (j == SPLIT - 1) ? slice_size : extended_slice_size) {
75                 slice_pixels[j][i] = pixels[i + j * slice_size];
76             }
77         }
78     }
79
80     // Obtain histogram from sliced pixels
81     loop_process: for (j = 0; j < SPLIT; j++) {
82         extract_histogram(slice_pixels[j], histogram[j], (j == SPLIT - 1) ? (IMG_HEIGHT / SPLIT - 1) : (IMG_HEIGHT / SPLIT + 1));
83     }
84
85     // Calculate the norm
86     uint16_t temp;
87     for (i = 0; i < HISTOGRAM_WIDTH; i++) {
88         loop_gather1: for (j = 0; j < SPLIT; j++) {
89             temp = (j == 0) ? histogram[0][i] : temp + histogram[j][i];
90         }
91         histogram[0][i] = temp;
92         histogram_product[i] = histogram[0][i] * histogram[0][i];
93         acum += histogram_product[i];
94     }
95     float norm = fast_sqrt(acum);
96
97     // Normalize the histogram
98     for (i = 0; i < HISTOGRAM_WIDTH; i++) {
99         feature[i] = histogram[0][i] / norm;
100     }
101 }

```

```

102
103 void Classifier::SVM(float features[HISTOGRAM_WIDTH], uint8_t *prediction) {
104
105     uint8_t k;
106     uint16_t i;
107     float predictors[HISTOGRAM_WIDTH];
108     float b;
109     float z[HISTOGRAM_WIDTH];
110     float acum[TYPES] = {0};
111     float predicted;
112     float last_predicted = -100000;
113
114     loop_histogram: for (i = 0; i < HISTOGRAM_WIDTH; i++){
115         loop_types1: for (k = 0; k < TYPES; k++){
116             predictors[i] = (features[i] - mu[k][i])/sigma[k][i];
117             z[i] = predictors[i]/scale[k];
118             acum[k] += z[i] * beta[k][i];
119         }
120     }
121
122     loop_types2: for (k = 0; k < TYPES; k++){
123         predicted = acum[k] + bias[k];
124
125         if (predicted > last_predicted)
126         {
127             *prediction = k;
128             last_predicted = predicted;
129         }
130     }
131 }
132
133 //Similar to Newton-Raphson
134 float Classifier::fast_sqrt(float number) {
135     union {
136         float f;
137         uint32_t i;
138     } conv = { number };
139
140     conv.i = 0x1fbd1df5 + (conv.i >> 1);
141     conv.f = 0.5f * (conv.f + number / conv.f);
142
143     return conv.f;
144 }

```

```
1  #ifndef TESTBENCH_H
2  #define TESTBENCH_H
3
4  #include <systemc.h>
5  #include "classifier.h"
6  #include <fstream>
7
8  #define IMG_WIDTH 28
9  #define IMG_HEIGHT 28
10
11 SC_MODULE(Testbench) {
12     sc_signal<bool> clk;
13     sc_signal<bool> reset;
14     sc_signal<bool> start;
15     sc_signal<uint8_t> pixels[IMG_WIDTH * IMG_HEIGHT];
16     sc_signal<uint8_t> prediction;
17
18     Classifier *classifier;
19
20     void run();
21     void load_test_image(const char *filename);
22
23     SC_CTOR(Testbench) {
24         classifier = new Classifier("classifier");
25         classifier->clk(clk);
26         classifier->reset(reset);
27         classifier->start(start);
28         for (int i = 0; i < IMG_WIDTH * IMG_HEIGHT; i++) {
29             classifier->pixels[i](pixels[i]);
30         }
31         classifier->prediction(prediction);
32
33         SC_THREAD(run);
34     }
35 };
36
37 #endif
```

```

1  //////////////////////////////////
2  // TESTBENCH.CPP //
3  //////////////////////////////////
4
5  #include "testbench.h"
6
7  // Thread for global execution
8  void Testbench::run() {
9      int test = 0;
10     reset = true;
11     start = false;
12     wait(10, SC_NS);
13
14     reset = false;
15     wait(10, SC_NS);
16
17     // Load an image
18     load_test_image("000000.raw");
19
20     start = true;
21     wait(10, SC_NS);
22     cout << "Prediction: " << (int)prediction.read() << endl;
23     start = false;
24
25     wait(100, SC_NS);
26
27     sc_stop();
28 }
29
30 // Intermediate memory
31 void Testbench::load_test_image(const char *filename) {
32
33     std::ifstream file(filename, std::ios::binary);
34
35     if (!file.is_open()) {
36         std::cerr << "Unable to open the file." << std::endl;
37         sc_stop();
38     }
39
40     uint8_t buffer[IMG_WIDTH * IMG_HEIGHT];
41     file.read(reinterpret_cast<char *>(buffer), IMG_WIDTH * IMG_HEIGHT); //
42     file.close();
43
44     for (int i = 0; i < IMG_WIDTH * IMG_HEIGHT; i++) {
45         pixels[i] = buffer[i];
46     }
47 }

```

```
1  ////////////
2  //  MAIN.CPP  //
3  ////////////
4
5  #include <systemc.h>
6  #include "testbench.h"
7
8  int sc_main(int argc, char* argv[]) {
9      Testbench tb("tb");
10
11      sc_start();
12
13      return 0;
14  }
```